Cypress

Testes E2E no browser

E2E, API, Component Testing, comandos, POM e primeiro teste

Automação · JavaScript · Cypress

O que é o Cypress?

Framework de automação **end-to-end** que roda testes diretamente no browser — sem WebDriver, sem trocar de contexto.

- Escrito em **JavaScript** / TypeScript
- API fluente e legível (`cy.visit`, `cy.get`, `cy.click`)
- Time-travel debugging, screenshots e vídeos automáticos
- Suporte a E2E, Component Testing e API testing

Por que usar Cypress?

⚡

Setup rápido

Instalação em minutos com npm. Sem drivers externos.

🔍

Debug visual

Runner interativo mostra cada comando e o DOM em tempo real.

🛡️

Retries automáticos

Reexecuta assertions até passar ou estourar timeout.

📦

Ecossistema

Plugins para a11y, relatórios, intercept de API e CI.

Pré-requisitos

Ambiente

- **Node.js** 18+ (LTS recomendado)
- **npm**, yarn ou pnpm
- Um app web rodando localmente ou em staging
- Editor: VS Code / Cursor (opcional: extensão Cypress)

```
$ node --version
v20.x.x

$ npm --version
10.x.x
```

Instalação — Passo a passo

1. **Crie ou entre no projeto**

   `mkdir meu-projeto && cd meu-projeto && npm init -y`

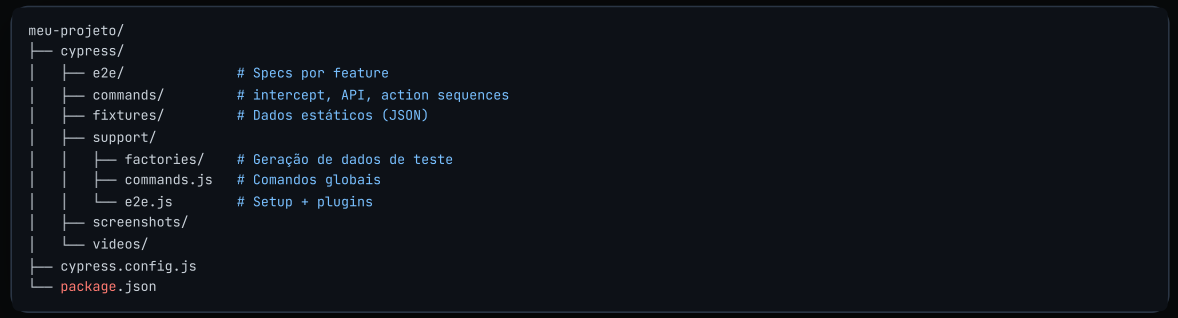
2. **Instale o Cypress**

   `npm install cypress --save-dev`

3. **Abra o Cypress pela primeira vez**

   `npx cypress open` — gera a pasta `cypress/` automaticamente

Estrutura do projeto



```
meu-projeto/
├── cypress/
│   ├── e2e/              # Specs por feature
│   ├── commands/         # intercept, API, action sequences
│   ├── fixtures/         # Dados estáticos (JSON)
│   ├── support/
│   │   ├── factories/    # Geração de dados de teste
│   │   ├── commands.js   # Comandos globais
│   │   └── e2e.js        # Setup + plugins
│   ├── screenshots/
│   └── videos/
├── cypress.config.js
└── package.json
```

- `commands/` — intercepts, seed API e fluxos compostos
- `factories/` — dados dinâmicos com Faker
- `e2e/*.cy.js` — um arquivo = um fluxo ou feature
- `support/commands.js` — `getByHook` e helpers globais

Configuração mínima

```
// cypress.config.js
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000',
    specPattern: 'cypress/e2e/**/*.cy.js',
    supportFile: 'cypress/support/e2e.js',
    viewportWidth: 1280,
    viewportHeight: 800,
  },
})
```

Com `baseUrl` definida, use `cy.visit('/')` em vez da URL completa.

Seu primeiro teste

```
// cypress/e2e/sign-in.cy.js
describe('Sign In', () => {
  it('should authenticate with valid credentials', () => {
    // Given: user is on sign-in page
    cy.visit('/sign-in')

    // When: user submits valid credentials
    cy.getByHook('email-input').type('user@example.com')
    cy.getByHook('password-input').type('Secret123!', { log: false })
    cy.getByHook('submit-btn').click()

    // Then: dashboard is visible
    cy.url().should('include', '/dashboard')
    cy.getByHook('welcome-message').should('be.visible')
  })
})
```

Convenção fictícia: atributo `data-cy-hook` + comando `cy.getByHook()`

Navegação & DOM

Comandos comuns em specs E2E — exemplos fictícios de um app "Acme Portal"

Comando	Uso típico
`cy.visit(path)`	Abrir sign-in, dashboard, páginas internas
`cy.get(selector)`	CSS, roles ARIA, `body`
`cy.getByHook(id)`	Custom — shorthand para `[data-cy-hook]`
`cy.url()`	Validar redirect após auth
`cy.title()`	Smoke — título da página
`cy.reload()`	Recarregar após mock de API
`cy.window()`	Inspecionar storage pós-login
`.find()`	Linhas de tabela, células, colunas

Interações na chain

Comando	Uso típico
`.type(text)`	Formulários, busca, filtros
`.click()`	Submit, tabs, modais, paginação
`.clear()`	Limpar campo antes de digitar
`.select(value)`	<select> nativo — timezone, role, env
`{ force: true }`	Elemento coberto por overlay
`{ log: false }`	Ocultar senha nos logs
`{ timeout: 5000 }`	Aguardar estado async
`body.type('{esc}')`	Fechar modal com Escape
`cy.contains(text)`	Encontrar botão/link por label visível
`.eq(n)`	Selecionar o n-ésimo elemento duplicado
`.scrollIntoView()`	Rolar até elemento fora do viewport
`.parent() / .siblings()`	Navegar no DOM relativo ao elemento
`.check() / .uncheck()`	Checkboxes e consent terms

Assertions

Assertion	Exemplo
`.should('be.visible')`	Modais, cards, toast, tabs
`.should('exist') / .not.exist`	Page root, elemento removido
`.should('contain.text', …)`	Mensagens, contadores, erros
`.should('have.length', n)`	Linhas de tabela, lista de items
`.should('have.attr', …)`	aria-selected, aria-hidden, placeholder
`.should('have.class', …)`	Nav item ativo
`.should('match', /regex/)`	Formato de data, percentual
`.and(…)`	Encadear assertions
`.its('status')`	Status code em `cy.request`
`.invoke('text')`	Extrair valor numérico de UI

cy.intercept — spy & mock

```
// Spy — observa chamada real
cy.intercept('POST', '/api/v1/auth/signin').as('signInApi')
SignInPage.submitCredentials(email, password)
cy.wait('@signInApi').its('response.statusCode').should('eq', 200)

// Mock — simula erro 401
cy.intercept('POST', '/api/v1/auth/signin', {
  statusCode: 401,
  body: { message: 'Invalid credentials' },
}).as('signInFail')
cy.wait('@signInFail')
SignInPage.shouldStayOnSignInPage()

// Inspect API payload
cy.intercept('POST', '/api/v1/users/invite').as('inviteApi')
UsersPage.sendInvite(name, email)
cy.get('@inviteApi').its('request.body.email').should('eq', email)
```

`.as('alias')` `cy.wait('@alias')` `cy.get('@alias')` `GET / PUT / PATCH` `regex URL`

cy.request & API testing

```
// Auth direto — mais rápido que UI
cy.request({
  method: 'POST',
  url: '/api/v1/auth/signin',
  body: { email, password },
})

// Chamada API autenticada (comando customizado)
cy.apiWithAuth({ method: 'GET', url: '/api/v1/users' })

// Health check
cy.request('/health').its('status').should('eq', 200)
cy.request({ url: '/api/v1/not-found', failOnStatusCode: false })
  .its('status').should('eq', 404)

// Validar shape da resposta
cy.assertResponseShape(user, { id: 'number', name: 'string' })
```

`cy.request` `cy.apiWithAuth` `cy.assertResponseShape` `failOnStatusCode: false` `onBeforeLoad`

Comandos customizados (E2E)

Definidos em `cypress/support/commands.js` — convenção `data-cy-hook`

```
// cypress/support/commands.js
Cypress.Commands.add('getByHook', (hook) =>
  cy.get(`[data-cy-hook="${hook}"]`)
)
Cypress.Commands.add('assertHookMissing', (hook) =>
  cy.get(`[data-cy-hook="${hook}"]`).should('not.exist')
)
```

Comando	Quando usar
`cy.getByHook(id)`	Selector padrão da suíte
`cy.assertHookMissing(id)`	Elemento ausente no DOM
`cy.signInViaUI()`	Auth via formulário — testa fluxo
`cy.signInViaAPI()`	Auth programática — beforeEach rápido
`cy.visitAsUser(path)`	Navegar já autenticado
`cy.apiWithAuth(opts)`	Chamada HTTP com Bearer token
`cy.assertResponseShape(obj, schema)`	Validar tipos da resposta
`cy.getTableRows(hook)`	Linhas visíveis de tabela
`cy.getTableCell(rowId, field)`	Célula por id de linha

Fixture, stub & eventos

```
// Fixtures — dados deterministicos
cy.fixture('users/valid-user').as('user')
cy.fixture('users/invite-payload').as('invite')

// cy.wrap — iterar elementos
cy.getTableRows().each(($row) => {
  cy.wrap($row)
    .find('[data-cy-hook="role-badge"]')
    .should('exist')
})

// cy.stub — controlar dialogs nativos
cy.window().then((win) => {
  cy.stub(win, 'confirm').returns(false).as('confirmStub')
})
cy.get('@confirmStub').should('have.been.called')
```

Eventos do browser

- `cy.on('window:alert')` — captura alert
- `cy.on('window:confirm')` — confirm true/false
- `Cypress.on('uncaught:exception')` — ignora erros do app

Plugins úteis

- `cypress-axe` — acessibilidade
- `cypress-mochawesome-reporter` — relatórios CI

Page Object Model (POM)

Padrão que **encapsula** seletores, ações e assertions de cada página em uma classe — o spec fica legível e mudanças de UI ficam isoladas.

Spec (.cy.js)

- Descreve o *cenário*
- Given / When / Then
- Sem seletores espalhados

→

Page Object

- Selectors (`getByHook`)
- Actions (`signInWith`)
- Assertions (`shouldBeLoaded`)

cypress/pages/ — SignInPage · DashboardPage · UsersPage · SettingsPage

Anatomia de um Page Object

```
// cypress/pages/SignInPage.js
class SignInPage {
  emailInput()  { return cy.getByHook('email-input') }
  submitBtn()   { return cy.getByHook('submit-btn') }

  visit() {
    cy.visit('/sign-in')
    return this
  }

  signInWith(email, password) {
    this.emailInput().clear().type(email)
    this.passwordInput().clear().type(password, { log: false })
    this.submitBtn().click()
    return this
  }

  shouldReachDashboard() {
    cy.getByHook('page-dashboard').should('exist')
    cy.url().should('include', '/dashboard')
    return this
  }
}

export default new SignInPage()
```

POM no spec

```
// cypress/e2e/auth/sign-in.cy.js
import SignInPage from '../../pages/SignInPage'

describe('Authentication', () => {
  beforeEach(() => SignInPage.visit())

  it('signs in via UI and reaches dashboard', () => {
    SignInPage
      .signInWith('user@example.com', 'Secret123!')
      .shouldReachDashboard()
  })

  it('signs in with API mode + intercept', () => {
    cy.intercept('POST', '/api/v1/auth/signin').as('signInApi')

    SignInPage
      .enableApiMode()
      .signInWith('user@example.com', 'Secret123!')

    cy.wait('@signInApi').its('response.statusCode').should('eq', 200)
    SignInPage.shouldReachDashboard()
  })
})
```

POM + beforeEach pattern

```
// dashboard.cy.js
import DashboardPage from '../../pages/DashboardPage'

beforeEach(() => {
  cy.signInViaAPI()
  DashboardPage.shouldBeLoaded()
})

it('shows metric cards', () => {
  DashboardPage.shouldShowAllMetrics()
  DashboardPage.metricValue('orders')
    .invoke('text').then(parseInt)
    .should('be.greaterThan', 0)
})
```

```
// users.cy.js
import UsersPage from '../../pages/UsersPage'

beforeEach(() => {
  cy.visitAsUser('/users')
  UsersPage.pageRoot().should('exist')
})

it('filters rows by name', () => {
  UsersPage.search('Jane')
  UsersPage.shouldShowRowCount(1)
  UsersPage.nameCell(1)
    .should('contain.text', 'Jane Doe')
})
```

`SignInPage.js` auth flow

`DashboardPage.js` metrics, actions

`UsersPage.js` table, invite, edit

`SettingsPage.js` forms, preferences

E2E Avançado — visão geral

Padrões de suítes E2E enterprise: sessão cacheada, seed de API, intercept com mutação de payload, wizard multi-step e relatório BDD.

Setup

`cy.session` · seed API · multi-env

Dados

Factories · Faker · interfaces TS

Rede

Intercept + `req.reply` · helpers nomeados

DX

`cy.section/step` · tags · enums de timeout

cy.session — login cacheado

```
// before() — login uma vez, reutiliza em todos os specs
before(() => {
  cy.seedAuthToken(user, pass)
  cy.seedComplianceStatus('NOT_STARTED')

  cy.session([user, pass], () => {
    cy.signInViaUI(user, pass)
    cy.url().should('not.include', '/sign-in')
    cy.skipOnboardingTour()
    cy.getByHook('header-profile-btn').should('be.visible')
  }, {
    validate() {
      cy.getCookies().then((cookies) => {
        if (!cookies.length) throw new Error('Session expired')
        cy.validateSession(Cypress.env('baseUrl'))
      })
    },
  })
})

beforeEach(() => {
  cy.visit(Cypress.env('baseUrl'))
  ProfilePage.open()
})
```

cacheAcrossSpecs | validate() | before vs beforeEach

Seed de estado via API

```
// cypress/commands/api-setup.js — preparar estado antes da UI
Cypress.Commands.add('seedAuthToken', (user, pass) => {
  cy.apiSignIn(user, pass).then(({ body }) => {
    Cypress.env('accessToken', body.token)
  })
})

Cypress.Commands.add('seedComplianceStatus', (status) => {
  cy.apiWithAuth({
    method: 'PUT',
    url: '/api/v1/compliance/sections',
    body: { personal: status, employment: status },
  })
})

Cypress.Commands.add('cancelPendingTickets', (userId) => {
  cy.apiWithAuth({
    method: 'DELETE',
    url: `/api/v1/users/${userId}/pending-changes`,
  })
})

// beforeEach — estado limpo por teste
beforeEach(() => {
  cy.cancelPendingTickets(testUser.id)
  cy.interceptGetProfile().as('getProfile')
  ProfilePage.open()
  cy.wait('@getProfile')
```

Intercept — mutar response

```
// Forçar telefone não verificado sem alterar backend
cy.interceptGetProfileAndPatch((body) => {
  body.profile.phoneVerified = false
  body.profile.phone = '+1 555 0100'
}).as('getProfile')

// Resetar pending changes no GET
cy.intercept('GET', '/api/v1/profile/basic', (req) => {
  req.reply(({ body }) => {
    body.pendingProfile.displayName = null
    body.pendingProfile.dateOfBirth = null
  })
}).as('resetPending')

// Mock de lookup countries
cy.intercept('GET', '/api/v1/lookups/countries', {
  fixture: 'lookups/countries',
}).as('countries')

cy.wait('@getProfile')
cy.getByHook('phone-verify-btn').should('be.visible')
```

Helpers nomeados: `interceptGetProfile()` · `interceptPatchProfile()` · `interceptGetCompliance()`

Factories & Faker

```
// cypress/support/factories/ContactFactory.ts
import { faker } from '@faker-js/faker'

export class ContactFactory {
  static createEmail({ verified = true } = {}) {
    return {
      email: faker.internet.email(),
      isVerified: verified,
    }
  }

  static createPhone({ country = 'CA' } = {}) {
    return {
      number: faker.phone.number(),
      country,
      isVerified: false,
    }
  }
}

// No spec:
const email = ContactFactory.createEmail({ verified: false })
cy.interceptGetProfileAndPatch((body) => {
  body.profile.email = email.email
  body.profile.isEmailVerified = email.isVerified
})
```

`@faker-js/faker` `interfaces TS` `enums de status`

Comandos compostos (actions)

Sequências de UI encapsuladas — o spec descreve o cenário, não cada clique.

```
// cypress/commands/actions.js
Cypress.Commands.add('fillDateOfBirth', (day, month, year) => {
  cy.openDropdown('dob-month', month)
  cy.getByHook('dob-day').clear().type(day)
  cy.getByHook('dob-year').clear().type(String(year))
})

Cypress.Commands.add('completePhoneVerification', (phone, code) => {
  cy.getByHook('edit-phone-btn').click()
  cy.getByHook('phone-input').find('input[type="tel"]').type(phone)
  cy.interceptPhoneValidate().as('validate')
  cy.clickHook('dialog-confirm')
  cy.getByHook('verify-via-sms').click()
  cy.clickHook('dialog-confirm')
  cy.wait('@validate').then(({ body }) => {
    cy.get('#otp-code').type(code ?? body.code)
    cy.clickHook('dialog-confirm')
  })
})

// No spec:
cy.section('PHONE VERIFICATION')
cy.step('Update and verify phone number')
cy.completePhoneVerification('5551234567')
cy.get('span:contains("Verified")').should('be.visible')
```

Wizard multi-step

```
describe('Annual compliance review', () => {
  it('completes all wizard sections', () => {
    cy.contains('button', 'Start review').click()

    // Section 1 — Personal
    cy.section('PERSONAL INFO')
    cy.editProfileField('display-name', 'Alex', 'M', 'Smith')
    cy.getByHook('wizard-next').click()
    cy.getByHook('step-personal').should('have.class', 'step-completed')

    // Section 2 — Employment
    cy.section('EMPLOYMENT')
    cy.fillEmploymentDetails({ status: 'Employed', title: 'Analyst' })
    cy.getByHook('wizard-next').click()
    cy.getByHook('step-employment').should('have.class', 'step-completed')

    // Final confirmation
    cy.section('CONFIRMATION')
    cy.getByHook('terms-checkbox').eq(0).check()
    cy.getByHook('terms-checkbox').eq(1).check()
    cy.getByHook('wizard-next').click()
    cy.contains('button', 'Done').click()
  })
})
```

cy.section & cy.step

Plugin `cypress-plugin-steps` — steps visíveis no runner e no relatório Mochawesome.

```
it('updates email and creates back-office ticket', () => {
  cy.section('Setup intercepts')
  cy.interceptPatchProfile().as('patchProfile')
  cy.interceptGetProfile().as('getProfile')

  cy.section('Open edit modal')
  cy.step('Click edit button')
  cy.get('button[data-cy-hook="edit-email-btn"]').click()

  cy.section('Submit change')
  cy.step('Type new email and confirm')
  cy.getByHook('email-input').type('new@example.com')
  cy.clickHook('dialog-confirm')

  cy.section('Assert API payload')
  cy.step('Validate PATCH body')
  cy.wait('@patchProfile')
    .its('request.body.email')
    .should('eq', 'new@example.com')
})
```

Tags — filtrar specs

```
// Plugin @bahmutov/cy-grep
describe('Profile — Email', { tags: '@regression' }, () => {
  it('creates ticket on name change', { tags: '@smoke @critical' }, () => {
    // ...
  })

  it.skip('handles edge case', { tags: '@wip' }, () => {
    // ...
  })
})

// Executar só smoke:
// npx cypress run --env grepTags=@smoke
// npx cypress run --env grepTags=@smoke+@critical
// grepFilterSpecs: true  → carrega só specs com a tag
```

Shadow DOM & config avançada

```
// Page Object com Shadow DOM
class ProfilePage {
  shadowFind(hook) {
    return cy.get('profile-app-root')
      .shadow()
      .find(`[data-cy-hook="${hook}"]`)
  }

  openEditModal(field) {
    this.shadowFind(`edit-${field}-btn`).click({ force: true })
  }
}
```

```
// cypress.config.js (trechos)
module.exports = defineConfig({
  includeShadowDom: true,
  chromeWebSecurity: false,
  testIsolation: false,       // state shared in suite
  pageLoadTimeout: 120000,
  defaultCommandTimeout: 60000,
  experimentalMemoryManagement: true,
  numTestsKeptInMemory: 1,
  blockHosts: ['*.analytics.com', '*.ads.net'],
  e2e: {
    setupNodeEvents(on, config) {
      require('@bahmutov/cy-grep/src/plugin')(config)
      on('task', { seedDb, resetDb })
      return config
    },
  },
})
```

cy.task & multi-ambiente

```
// Node task — seed direto no banco (CI/staging)
cy.task('resetUserProfile', { userId: 'abc-123' })
cy.task('setComplianceFlag', { userId: 'abc-123', flag: 'REVIEW_DUE' })

// Ambientes via config
Cypress.Commands.add('setEnvironment', (env) => {
  Cypress.env('ENV', env)  // 'staging' | 'production'
})

Cypress.Commands.add('visitPage', (path = '/') => {
  cy.getBaseUrl().then((base) => cy.visit(`${base}${path}`))
})

// CLI: npx cypress run --env ENV=staging,version=staging
```

Pasta / artefato	Responsabilidade
`commands/interceptions.js`	Helpers de `cy.intercept`
`commands/api-setup.js`	Seed e cleanup via `cy.request`
`commands/actions.js`	Fluxos compostos de UI
`support/factories/`	Dados dinâmicos
`support/@enums/`	Status, timeouts, viewports
`support/loadConfig.js`	Config + secrets por ambiente

API Testing puro `cy.request`

Cypress também testa **APIs REST** sem browser — ideal para rule engines, microserviços e contratos de dados.

E2E (UI)

`cy.visit` + interações DOM

API

`cy.request` — sem UI, foco em contrato

- Specs por domínio: `e2e/personal/`, `e2e/tax/`, `e2e/financial/`
- Commands por serviço: `personalCommands`, `taxCommands`
- Utilities: builders, patterns, retry, JSON Patch utils

Arquitetura dual — write + read

Rules API (write)

- `PATCH /users/{id}`
- JSON Patch body
- Retorna `{ isSuccess: true }`

→

Profile API (read)

- `GET /users/{id}/personal`
- Valida persistência
- Retry com backoff

```
// Fluxo padrão
cy.step('Patch display name via Rules API')
cy.patchDisplayNameViaRules(userId, patches).then((res) => {
  expect(res.status).to.eq(200)
  expect(res.body).to.deep.eq({ isSuccess: true })
})

cy.step('Validate via Profile API')
retryProfileValidation(userId, 'name', 'profile', expectedValues)
```

JSON Patch (RFC 6902)

```
// Content-Type: application/json-patch+json
const patches = new JsonPatchBuilder()
  .replace('/personal/firstName', 'Alex')
  .replace('/personal/middleName', null)
  .replace('/personal/lastName', 'Smith')
  .build()

// Resultado:
// [
//   { op: 'replace', path: '/personal/firstName', value: 'Alex' },
//   { op: 'replace', path: '/personal/middleName', value: null },
//   { op: 'replace', path: '/personal/lastName', value: 'Smith' },
// ]

// Helpers utilitários
extractPatchValues(patches)       // { firstName: 'Alex', ... }
modifyPatchField(patches, path, v) // altera um campo para teste negativo
removePatchField(patches, path)    // remove operação do array
```

`replace` `add` `remove` `JsonPatchBuilder`

Autenticação OAuth

```
// before() — token de serviço (client credentials)
before(() => cy.setServiceToken())

// Token de negócio — escopo de usuário específico
cy.setBusinessToken(userId)

// Headers em cada chamada
const headers = {
  Authorization: `Bearer ${token}`,
  'Content-Type': 'application/json-patch+json',
  'Correlation-Id': generateUuid(),   // rastreio por teste
}

// Secrets via CI/CD ou Secret Manager — nunca hardcode
cy.getServiceCredentials().then(({ client_id, client_secret }) => {
  cy.apiSignIn({ client_id, client_secret, grant_type: 'client_credentials' })
})
```

`log: false` em requests com credenciais — oculta secrets nos logs do Cypress.

patch vs tryPatch

```
// patch — espera sucesso (200)
cy.patchDisplayNameViaRules(userId, patches)
  .then((res) => {
    expect(res.status).to.eq(200)
  })

// tryPatch — aceita falha (validação)
// failOnStatusCode: false
cy.tryPatchDisplayNameViaRules(patches)
  .then((res) => {
    expect(res.status).to.be.oneOf([400, 422])
  })
```

Quando usar

- **patch** — happy path, fluxo completo
- **tryPatch** — testes negativos (campo nulo, formato inválido, tamanho máximo)

Headers dinâmicos

Array body → `application/json-patch+json`

Object body → `application/json`

Retry com backoff exponencial

Após PATCH, a leitura pode demorar — retry até o Profile API refletir os dados.

```ts
// retryUtils.ts
const DEFAULT_CONFIG = {
  maxRetries: 5,
  baseDelay: 1000,
  maxDelay: 10000,
  exponentialBackoff: true,
}

export function retryProfileValidation(userId, component, node, expected) {
  const attempt = (retriesLeft) => {
    cy.getProfilePersonal(userId, component, node).then((res) => {
      try {
        expect(res.status).to.eq(200)
        expect(res.body[node].firstName).to.eq(expected.firstName)
      } catch (err) {
        if (retriesLeft <= 1) throw err
        const delay = 1000 * Math.pow(2, 5 - retriesLeft)
        cy.wait(Math.min(delay, 10000))
        attempt(retriesLeft - 1)
      }
    })
  }
  attempt(5)
}
```

Test Suite Builder

```
// Encapsula happy path + validation por domínio
class DisplayNameTestBuilder extends PersonalDataTestSuiteBuilder {
  constructor() {
    super('Display Name', 'name',
      'patchDisplayNameViaRules',
      'tryPatchDisplayNameViaRules',
      'displayNameBody')
  }

  buildValidationTests() {
    context('Invalid Characters', () => {
      TestPatterns.generateInvalidCharacterTests([...], attrs, 'tryPatch...')
    })
    context('Mandatory Fields', () => {
      TestPatterns.generateMandatoryFieldTests(['firstName', 'lastName'], ...)
    })
    context('Max Length', () => {
      TestPatterns.generateMaxLengthTests(attrs, ..., 255)
    })
  }
}

// Uso no spec:
const builder = new DisplayNameTestBuilder()
builder.buildCompleteSuite(
  () => builder.buildHappyPathTests(),
  () => builder.buildValidationTests(),
  () => TestDataFactory.createDisplayNamePatch()
```

Test Patterns — gerar cenários

```
// Happy path parametrizado por estado de identidade
[false, true].forEach((isVerified) => {
  context(`Happy Path | verified === ${isVerified}`, () => {
    beforeEach(() => {
      setUserByVerificationStatus(isVerified)
      cy.wrap(TestDataFactory.createPatch()).as('patchBody')
    })

    it('[TC-0001] patches display name successfully', () => {
      cy.get('@uuid').then((userId) => {
        cy.get('@patchBody').then((patches) => {
          TestPatterns.executeSuccessfulPatchFlow(userId, patches, ...)
        })
      })
    })
  })
})

// Idempotency — PATCH duplicado
it('[TC-0010] accepts identical patch twice', () => {
  cy.patchViaRules(userId, patches).its('status').should('eq', 200)
  cy.patchViaRules(userId, patches).its('status').should('eq', 200)
})
```

Generators & Test Data Factory

```
// support/@generators/ — dados válidos por país/regra
generateUuid()
generateTaxId('CA')          // formato doméstico
generateTaxId('US')          // formato estrangeiro
generateDateOfBirth({ age: 25 })

// support/utilities/testDataFactory.ts
class DisplayNameTestDataFactory {
  static createCompletePatch() {
    return createDisplayNamePatch(
      faker.person.firstName(),
      faker.person.middleName(),
      faker.person.lastName(),
    )
  }
}

// Fixtures por país para endereço
cy.fixture('addresses/domestic').then((addr) => {
  const patch = convertAddressToPatch(addr)
  cy.patchAddressViaRules(userId, patch)
})
```

Spec completo — API contract test

```
describe('Display Name', () => {
  before(() => cy.setServiceToken())

  context('Validation', () => {
    beforeEach(() => {
      setUserByVerificationStatus(false)
      cy.wrap(TestDataFactory.createPatch()).as('patchBody')
    })

    it('[TC-0003] rejects numbers in firstName', () => {
      cy.get('@patchBody').then((body) => {
        const invalid = modifyPatchField(body, '/personal/firstName', '123')
        cy.tryPatchDisplayNameViaRules(invalid)
          .its('status').should('be.oneOf', [400, 422])
      })
    })

    it('[TC-0009] rejects null firstName', () => {
      cy.get('@patchBody').then((body) => {
        const invalid = modifyPatchField(body, '/personal/firstName', null)
        cy.tryPatchDisplayNameViaRules(invalid)
          .its('status').should('be.oneOf', [400, 422])
      })
    })
  })
})
```

Infra & relatórios API

Recurso	Uso
`loadConfigWithSecrets()`	Config + secrets por ambiente (CI/CD)
`cy.task('logJson', body)`	Debug de response no terminal CI
`logTestTitle()`	Rastreia IDs de teste em arquivo por spec
`cypress-mochawesome-reporter`	HTML + JSON com steps
`retries: { runMode: 2 }`	Reexecuta spec em flake de rede
`testIsolation: false`	Compartilha token entre tests do describe

	E2E UI	API	Component
Foco	Fluxo usuário	Contrato REST	UI isolada
Comando core	`cy.visit`	`cy.request`	`cy.mount`
Dados	Fixtures UI	JSON Patch + Factory	Props mock
Validação	DOM assertions	Status + body deep.eq	Hook visible
POM	Page classes	Builders + Patterns	Hook maps

Component Testing `cy.mount`

Testa um **componente isolado** no browser — sem navegar pela app inteira. Ideal para libs de UI, formulários e modais.

E2E

`cy.visit('/app')` — fluxo completo

Component

`cy.mount(Component)` — isolado e rápido

- Spec ao lado do componente: `*.cy.tsx` ou `*.cy.ts`
- Config: `component.devServer` no `cypress.config`
- Props, providers e context injetados no mount

cy.mount — setup do componente

```tsx
// UserBadge.cy.tsx — React (exemplo fictício)
import { UserBadge } from './UserBadge'
import { AppProvider } from '../providers/AppProvider'

describe('UserBadge', () => {
  beforeEach(() => {
    cy.mount(
      <AppProvider locale="en">
        <UserBadge user={{ name: 'Alex', verified: false }} />
      </AppProvider>
    )
  })

  it('shows verify action when user is unverified', () => {
    cy.getByHook('verify-action').should('be.visible')
  })
})

// Angular equivalent uses cy.mount(Component, { imports, providers })
```

`cy.mount` `providers / imports` `componentProperties` `createOutputSpy()`

POM — Hook Maps (Elements)

Em component tests, o POM costuma ser um **mapa de hooks** exportado de `cypress/support/elements/`.

```ts
// cypress/support/elements/dialog.ts
export const DIALOG = {
  confirmBtn:  { hook: 'dialog-confirm' },
  cancelBtn:   { hook: 'dialog-cancel' },
  title:       { hook: 'dialog-title' },
  errorIcon:   { hook: 'dialog-error-icon' },
}

// cypress/support/elements/user-form.ts
export const USER_FORM = {
  firstName:   { hook: 'input-first-name' },
  lastName:    { hook: 'input-last-name' },
  errorMsg:    { hook: 'field-error-message' },
  helpText:    { hook: 'field-help-text' },
}

// No spec:
cy.getByHook(DIALOG.confirmBtn.hook).click()
cy.getByHook(USER_FORM.firstName.hook).type('Alex')
cy.assertHookVisible(USER_FORM.helpText.hook)
```

`shared.ts` radios, tooltips, overlay

`dialog.ts` modais, botões, erros

`user-form.ts` inputs, labels, validação

`data-table.ts` rows, sort, pagination

Seletores — comandos customizados

Comando	Descrição
`cy.getByHook(value)`	`[data-cy-hook="…"]` — selector principal
`cy.assertHookVisible(hook)`	Shorthand `.should('be.visible')`
`cy.assertHookMissing(hook)`	Elemento removido do DOM
`cy.clickHook(hook)`	get + click em um passo
`cy.get(selector)`	Fallback CSS quando necessário
`.find()`	Filhos dentro do componente montado
`.focus().blur()`	Disparar validação on blur

```
// Implementação sugerida
Cypress.Commands.add('getByHook', (hook) ⇒
  cy.get(`[data-cy-hook="${hook}"]`, { timeout: 10000 })
)
```

Design system — interações customizadas

Comando	Descrição
`cy.pickRadio(hook)`	Seleciona radio button por hook
`cy.openDropdown(hook, label)`	Abre dropdown e escolhe opção
`cy.openDropdownAt(hook, index)`	Seleciona opção por índice
`cy.closeDropdownOverlay()`	Fecha via backdrop/overlay
`cy.selectMenuItem(hook, label)`	Menu contextual por texto
`cy.clearField(hook)`	Limpa input associado ao hook
`cy.assertPlaceholder(hook, text)`	Valida placeholder de input

Encapsulam o design system — o spec não conhece classes CSS internas.

cy.intercept & mocks

```
// Spy — valida payload
cy.intercept('PATCH', '/api/v1/profile', { statusCode: 204 }).as('saveProfile')
cy.getByHook(USER_FORM.firstName.hook).type('Alex')
cy.clickHook(DIALOG.confirmBtn.hook)
cy.get('@saveProfile').its('request.body.firstName').should('eq', 'Alex')

// Mock com fixture estática
cy.mockApiGet('users/empty-list', '/api/v1/users')
cy.mockApiGet('lookups/countries', '/api/v1/lookups/countries')

// Helper reutilizável
cy.stubPatchResource('profile').as('patchProfile')
cy.wait('@patchProfile').its('response.statusCode').should('eq', 204)

// Alterar headers no voo
cy.intercept('POST', '/api/v1/verify', ({ headers }) => {
  headers['X-Test-Bypass-RateLimit'] = 'true'
})
```

Spec completo — dialog com validação

```tsx
// ConfirmDialog.cy.tsx (exemplo fictício)
describe('ConfirmDialog', () => {
  beforeEach(() => cy.mount(<ConfirmDialog open />))

  it('shows optional field when user picks Yes', () => {
    cy.pickRadio(SHARED.yesOption.hook)
    cy.assertHookVisible(USER_FORM.documentId.hook)
  })

  it('hides optional field when user picks No', () => {
    cy.pickRadio(SHARED.noOption.hook)
    cy.assertHookMissing(USER_FORM.documentId.hook)
  })

  it('shows validation error on empty blur', () => {
    cy.getByHook(USER_FORM.documentId.hook).click()
    cy.getByHook(DIALOG.title.hook).click()   // trigger blur
    cy.getByHook(DIALOG.errorIcon.hook).should('be.visible')
    cy.getByHook(USER_FORM.errorMsg.hook)
      .should('contain.text', 'This field is mandatory')
  })

  it('submits successfully', () => {
    cy.intercept('PATCH', '/api/v1/profile', { statusCode: 204 }).as('save')
    cy.getByHook(USER_FORM.documentId.hook).type('ABC-12345')
    cy.clickHook(DIALOG.confirmBtn.hook)
    cy.get('@save').its('response.statusCode').should('eq', 204)
  })
```

Parametrização de testes

```
function mountUserForm(props = {}) {
  cy.mount(<UserForm {...props} />)
}

const cases = [
  {
    title: 'hides help text when firstNameHelp is omitted',
    props: { labels: { firstName: 'First name' } },
    missingHook: USER_FORM.firstNameHelp.hook,
  },
  {
    title: 'hides help text when lastNameHelp is omitted',
    props: { labels: { lastName: 'Last name' } },
    missingHook: USER_FORM.lastNameHelp.hook,
  },
]

cases.forEach(({ title, props, missingHook }) => {
  it(title, () => {
    mountUserForm(props)
    cy.getByHook(USER_FORM.firstName.hook).click().blur()
    cy.getByHook(missingHook).should('not.exist')
  })
})
```

Um `it()` por combinação de props — evita duplicar o spec inteiro.

Responsive, fixtures & utilitários

Comando	Uso em component tests
`cy.viewport(w, h)`	Mobile / tablet / desktop
`cy.fixture('viewports/desktop')`	Lista de resoluções de teste
`cy.fixture(…).as('alias')`	Payloads, lookups, schemas
`cy.wrap(value)`	Re-enfileirar valor ou elemento
`cy.get('@alias')`	Reusar elemento ou intercept
`createOutputSpy('onSave')`	Espiar callback/evento de output
`Cypress.on('uncaught:exception')`	Suprimir erro do framework no mount

Comandos customizados (Component)

Seletores	
`getByHook` · `assertHookVisible` · `assertHookMissing` · `clickHook`	Base — `data-cy-hook`
Design system	
`pickRadio` · `openDropdown` · `openDropdownAt` · `selectMenuItem`	Interações compostas
`clearField` · `assertPlaceholder` · `closeDropdownOverlay`	Inputs e overlays
Mock & Intercept	
`mockApiGet(fixture, url)` · `mockApiPost(…)`	Fixture-based intercept
`stubGetResource(name)` · `stubPatchResource(name)`	Helpers por recurso REST
`cy.intercept` · `cy.wait('@alias')` · `cy.get('@alias')`	Spy, mock, inspect body
Setup (specs integrados)	
`cy.session` · `cy.request` · `seedTestData()`	Estado entre specs E2E de componente
`formatDisplayValue()` · `randomFromFixture()`	Helpers de assertion e dados

E2E vs Component — quando usar

	Component (`cy.mount`)	E2E (`cy.visit`)
Escopo	Um componente + providers	App inteira + routing
Velocidade	Rápido, isolado	Mais lento, integração real
POM	Hook maps (`elements/`)	Page classes (`pages/`)
Seletor	`data-cy-hook`	`data-cy-hook` (mesma convenção)
API	`intercept` + fixtures mock	`cy.request` + session real
Ideal para	Validação, erros, i18n, responsivo	Fluxos cross-page, auth, smoke CI

```
npx cypress open --component
npx cypress run --component --spec "**/UserBadge.cy.tsx"
```

Modo interativo vs CI

🖥️ cypress open

Runner visual para desenvolver e debugar testes.

```
npx cypress open
```

🤖 cypress run

Headless — ideal para CI/CD e pipelines.

```
npx cypress run
npx cypress run --spec "cypress/e2e/sign-in.cy.js"
```

Adicione scripts no `package.json`: `"cy:open"` e `"cy:run"`

Boas práticas para começar

- **Page Objects** em `cypress/pages/` — selectors, actions e assertions separados
- `cy.getByHook` + `data-cy-hook` — convenção única em E2E e component tests
- `cy.signInViaAPI` no `beforeEach` — specs de feature mais rápidos
- `cy.intercept` para validar payloads e simular erros de API
- Fixtures em `cypress/fixtures/` — dados fictícios, sem PII real
- Um cenário por `it()` — falhas fáceis de diagnosticar
- `cy.session` + `validate()` — login cacheado entre specs
- `cy.section` / `cy.step` — relatório BDD legível no CI
- Tags com `@bahmutov/cy-grep` — rodar só `@smoke` ou `@critical`
- Intercept com `req.reply` — mutar response sem mudar backend

Próximos passos

Você já sabe instalar, configurar e escrever testes E2E e de componente com Cypress.

📚 **Documentação**
docs.cypress.io

🧩 **Component Testing**
Guia oficial cy.mount

🎓 **Cypress Learn**
Tutoriais e exercícios oficiais

← → slides | ↓ ↑ sub-slides | F tela cheia | ESC overview